Применение готовых моделей машинного обучения

Отчёт по серии заданий 1 по программной инженерии

Выполнил: _Лапаев Степан_ Группа: 11-412

Репозиторий: https://github.com/lmaoooo45/ml-engineering-1

1 Цель и постановка задачи

Цель работы: создать приложения с использованием готовых библиотек машинного обучения. Разработаны четыре программы: определение тональности русского текста, распознавание речи, классификация изображения и обнаружение объектов на видео. Для вычислений используются PyTorch и TensorFlow. Модели применяются с готовыми весами, дополнительное обучение не проводится.

Каждая программа принимает данные через командную строку и сохраняет результат в JSON. Для видео дополнительно создаётся AVI с рамками. Проверка проведена на учебных данных.

2 Анализ тональности текста

Задача состоит в отнесении короткой русской фразы к отрицательному, нейтральному или положительному классу. Выбрана модель cointegrated/rubert-tiny-sentiment-balanced, доступная через Transformers. Она удобна для этой задачи благодаря специализации на русском языке и небольшому размеру. Библиотека предоставляет совместимые средства загрузки модели и токенизатора [1].

В основе используется модель семейства BERT. Токены переводятся в векторы, а механизм внимания учитывает контекст. Классификатор формирует оценки тональности [2]. В приложении токенизатор подготавливает числовые входы, затем PyTorch выполняет модель в режиме применения без расчёта градиентов. Для оценок используется sigmoid, как в примере автора. Выбирается класс с максимальной оценкой. Эти оценки не обязаны суммироваться в единицу.

По карточке модели данные дообучения приведены к трём классам и сбалансированы по источникам и меткам. Среди источников перечислены

отзывы, сообщения социальных сетей, новости и тематические тексты [1]. Структура классификационных данных: текст и эталонная метка. Собственная учебная проверка хранится в CSV с полями text и label. Она содержит 15 примеров, по 5 на класс, и не используется для изменения весов.

Для проверки вычисляются accuracy и macro-F1. Accuracy представляет долю правильных ответов. Macro-F1 усредняет F1 отдельных классов с равными весами. F1 = 2TP / (2TP + FP + FN), где TP является числом верных обнаружений класса, FP ошибочными отнесениями к нему, FN пропущенными примерами класса.

Получено 14 верных ответов из 15: accuracy = 0.9333, macro-F1 = 0.9327. Фраза «Доставка опоздала товар сломан» отнесена к neutral вместо negative. Малый и специально составленный набор подтверждает работу программы, но не даёт надёжной оценки обобщающей способности. Ирония, неоднозначная формулировка и обрезка длинного текста могут приводить к ошибкам.

3 Распознавание речи

Программа преобразует короткую аудиозапись в текст. Используется многоязычная модель openai/whisper-tiny через Transformers и PyTorch. Версия tiny выбрана для небольшой учебной демонстрации. Процессор библиотеки готовит признаки звука и преобразует выходные токены в строку [3].

Whisper является Transformer с encoder и decoder. Encoder обрабатывает log-Mel спектрограмму, decoder последовательно формирует токены расшифровки. В исходной работе описано обучение семейства моделей на 680 тысячах часов многоязычных аудиоданных [4]. Схематическая запись датасета содержит аудиосигнал, расшифровку и сведения о языке или задаче. Обучающий корпус не загружается в данном проекте.

FFmpeg из пакета imageio-ffmpeg декодирует запись в один канал с частотой 16000 Гц. Затем процессор подготавливает признаки, а generate выполняет задачу transcribe. По умолчанию установлен русский язык; для

английского демонстрационного файла задан en. Программа рассчитана на запись до 30 секунд и явно отклоняет более длинную. Выход ограничен 256 новыми токенами. Для продолжительных записей нужна отдельная обработка фрагментов.

Качество оценивается метрикой WER = (S + D + I) / N, где S является числом замен слов, D удалений, I вставок, N количеством слов эталона. Перед сравнением текст приводится к нижнему регистру, пунктуация исключается, ё заменяется на е. Скорость характеризует RTF: время обработки, делённое на длительность записи.

На английском файле jfk.flac продолжительностью 11 секунд получена расшифровка без отличий по словам от подготовленного эталона: WER = 0. На итоговом запуске обработка заняла около 0.47 секунды, RTF около 0.043. Результат относится только к одному короткому примеру. Проверка русского языка на пользовательской записи пока не выполнена. Шум, паузы и необычные слова могут ухудшать распознавание; на тишине возможен выдуманный текст.

4 Классификация изображений

Задача состоит в определении класса изображения целиком. Используется MobileNetV2 из Keras Applications с TensorFlow и весами ImageNet. Преимущество библиотеки: готовая архитектура, загрузка весов и стандартные функции предобработки и перевода выходов в названия классов [5].

MobileNetV2 является свёрточной нейросетью. Для сокращения вычислений используются раздельные свёртки и блоки с расширением и сжатием признаков. Такая архитектура подходит для демонстрации при ограниченных вычислительных ресурсах [6]. Это не детектор: координаты отдельных объектов она не выдаёт.

Веса относятся к классификации по 1000 классам ImageNet ILSVRC. Структура данных включает изображение и его эталонный класс; обучающая и проверочная части используются авторами готовых весов [7]. В приложении фотография преобразуется в RGB 224 на 224, затем в массив с дополнительным измерением пакета. preprocess_input переводит пиксели

в диапазон от -1 до 1. После применения модели decode_predictions возвращает пять классов с максимальными оценками [5].

Основные метрики: top-1 accuracy и top-5 accuracy. Первая проверяет совпадение самого вероятного класса с эталоном, вторая наличие эталонного класса среди пяти ответов. На фото dog.jpg первым получен Samoyed с оценкой 0.5864; за ним Pomeranian, Arctic_fox, wallaby и keeshond. Известный класс samoyed присутствует на первом месте. Одна фотография является примером успешной классификации, а не оценкой точности на выборке.

Время подготовки входа и применения сети составило около 0.16 секунды. В него не включены загрузка модели и перевод чисел в названия классов. Программа ограничена классами ImageNet, прямое изменение размера и сложный фон могут ухудшить результат.

5 Детекция объектов на видео

Программа читает видео по кадрам и применяет SSDLite320 с MobileNetV3 из torchvision. Выбраны готовые веса COCO_V1. Библиотека PyTorch выполняет модель, а OpenCV обеспечивает чтение, нанесение рамок и запись видео. Готовый детектор возвращает классы, оценки и координаты, что позволяет сосредоточиться на обработке видеопотока.

MobileNetV3 выделяет признаки кадра, SSD формирует предложения рамок и классов на разных масштабах. Встроенная обработка отбирает итоговые обнаружения. Детектор использует внутренний размер 320 на 320. По документации весов число параметров составляет 3 440 060, а опубликованный box mAP на COCO-val2017 равен 21.3 [8]. Это значение авторов весов, оно не измерено на нашем видео.

COCO содержит изображения бытовых сцен с разметкой объектов [9]. Для детекции нужны изображение, класс и прямоугольник объекта. В JSON-аннотациях используются записи изображений, категорий и объектов с координатами рамок. Основной детекционный набор имеет 80 классов объектов. Список меток torchvision также включает фон и пропуски, поэтому его нельзя напрямую считать списком 80 последовательных индексов.

Кадр из OpenCV преобразуется из BGR в RGB, затем в тензор с пикселями от 0 до 1. Обнаружения, прошедшие заданный порог, сохраняются в JSON и рисуются на кадре. По умолчанию порог равен 0.5. Выходной AVI сохраняет исходный FPS и содержит только выбранный начальный фрагмент. Звуковая дорожка не переносится. Объекты между кадрами не связываются, поэтому нельзя вычислять число уникальных людей по сумме обнаружений.

Для оценки качества используют IoU, precision, recall и mAP. IoU является отношением площади пересечения рамок к площади их объединения. Precision оценивает долю верных обнаружений среди выданных, recall долю найденных эталонных объектов. mAP обобщает результаты по классам и условиям сопоставления. Такие вычисления требуют эталонной разметки. На демонстрационном видео она не подготовлена, поэтому собственное mAP не заявляется.

На первых 30 кадрах vtest.avi при пороге 0.5 не осталось обнаружений. Для демонстрации порог снижен до 0.25, что позволило получить рамки с более низкими оценками. Это увеличивает риск ложных обнаружений и само по себе не доказывает улучшение качества. Программа создала читаемый AVI и JSON. Исходная частота составляет 10 кадров в секунду. Итоговый цикл обработки занял около 1.18 секунды, скорость около 25.45 кадра в секунду. Это измерение функциональной демонстрации на CPU, а не гарантированная скорость для любого видео. Маленькие и перекрытые объекты могут пропускаться.

6 Реализация и проверка

Решения расположены в text_task.py, audio_task.py, image_task.py и video_task.py. common.py сохраняет JSON, metrics.py вычисляет метрики, download_samples.py получает демонстрационные файлы. В requirements.txt зафиксированы основные версии. Ноутбук colab_demo.ipynb содержит команды запуска и просмотр результатов.

Проверка выполнена 11 октября 2026 года в Windows с Python 3.12.2, PyTorch 2.6.0 CPU, torchvision 0.21.0, Transformers 4.49.0, TensorFlow 2.18.0 и Keras 3.8.0. Выполнение моделей проверено локально. Запуск в реальном

сеансе Google Colab пока не подтверждён. Время обработки взято из результатов отдельных запусков программ, без специального прогрева и усреднения по повторным запускам. Это приблизительные наблюдения, пригодные для демонстрации, но не для строгого сравнения производительности.

Все четыре программы завершились успешно. Отдельно проверяются вычисления метрик, читаемость JSON и число записанных кадров. При первом запуске необходим интернет для скачивания файлов и весов. Подготовленные результаты позволяют проверить выводы без повторного скачивания.

Для будущего API выбрана функция SentimentAnalyzer.predict. Она принимает строку и возвращает словарь. Модель можно загрузить при старте сервера один раз. В текущую серию заданий HTTP API не входит.

7 Вывод

Созданы четыре приложения на основе готовых моделей с использованием двух фреймворков. Реализованы подготовка входных данных, предсказание и сохранение результатов. Демонстрационные запуски подтвердили выполнение всех четырёх задач. Для более надёжной оценки нужны расширенный независимый набор текстов, несколько записей речи, размеченная подборка изображений и эталонные рамки на кадрах видео.

Источники

1. Карточка модели rubert tiny sentiment balanced
2. BERT Pre training of Deep Bidirectional Transformers
3. Карточка модели Whisper tiny
4. Robust Speech Recognition via Large Scale Weak Supervision
5. Keras MobileNetV2 исходный код
6. MobileNetV2 Inverted Residuals and Linear Bottlenecks
7. ImageNet ILSVRC 2012
8. Torchvision SSDLite320 MobileNetV3
9. COCO

Дата обращения к источникам: 11 октября 2026 года.